

REGINSIGHT AI · SOURCE-VERIFIED GUIDE

Application Flow from Inspection Evidence to Human Review

A practical map of how the running application receives a question, routes and retrieves evidence, creates and evaluates an assessment, persists each stage, and reaches an authenticated reviewer.

Scope: This guide follows the explicit AI assessment workflow in the Evidence workspace. The existing chatbot remains a separate path and is not silently routed through this workflow.

Prepared from the current project source: FastAPI, React, PostgreSQL/pgvector, a PostgreSQL job queue, LangGraph checkpoints, and local Ollama model calls.

Local application URL: <http://127.0.0.1:8018>

1. Whole-application picture

The web application has two related but distinct review/analysis capabilities. The normal website serves dashboards, inspection and evidence browsing, analytics, chatbot/agent features, and human review of observation classifications. The AI assessment workflow is an explicit user-started path embedded under the selected observation in Evidence.

```
Windows launcher
  Start-RegInsight.ps1
    |
    ├── Docker Compose: PostgreSQL + pgvector
    ├── migrate/import: schema + supplied dataset
    ├── app: FastAPI + built React frontend (:8018)
    └── worker: durable background-job consumer

Browser → React Evidence → POST /api/workflows
  → PostgreSQL workflow + queued job
  → worker → WorkflowService → LangGraph stage graph
  → retrieve evidence / existing analytics services
  → specialist generation → independent judge evaluation
  → PostgreSQL human-review gate
  → authenticated reviewer decision → resume worker
  → final response (or revision/rejection)
```

Startup and runtime roles

Role	What it does	Source location
Launcher / Compose	Builds app images, starts PostgreSQL, runs database migrations and imports prepared data, then starts API and worker services.	Start-RegInsight.ps1; docker-compose.yml
FastAPI app	Serves JSON APIs and compiled frontend; initializes the existing intelligence/chatbot services and workflow service.	backend/api/main.py — create_app(), lifespan
Workflow worker	Claims durable jobs from PostgreSQL, runs workflows, renews leases, retries selected transient errors.	backend/workflows/worker.py — main(), run_once()
PostgreSQL	Canonical runtime database, workflow/event/review/job persistence, pgvector embeddings and full-text retrieval.	docker-compose.yml; backend/database; backend/rag/repository.py
Ollama	Local embedding, specialist generation, and judge model endpoint. AI-model outage can disable workflow creation while deterministic analytics remain.	backend/rag/embeddings.py; backend/libraries/structured_llm.py

Important: The local database password in `.env` is a secret. Keep it out of screenshots, chat, source control, and this document. The project ignores `.env`.

2. Entry point: selected observation to queued workflow

1 — Select the source record. In the Evidence workspace, the selected observation supplies its original text, observation ID, and intelligence run ID. `frontend/src/reginsight/Evidence.jsx` renders `WorkflowReview`.

2 — Start an assessment. The user edits the question if needed and presses "Start AI assessment." `frontend/src/reginsight/WorkflowReview.jsx:start()` sends the question, `observation_id`, `run_id`, and `require_review: true` through the shared request helper.

3 — Validate/authenticate the API request. Security middleware authenticates the signed-in session, checks host/origin and request bounds, and assigns a request ID. The route validates `WorkflowRequest`; this workflow requires an authenticated API user.

4 — Persist before processing. `POST /api/workflows` calls `WorkflowService.create()`. It creates IDs and a `RECEIVED` state; repository inserts the workflow/event, and the same database transaction enqueues a uniquely keyed start job. The API responds `202 Accepted`.

5 — Background execution. The browser periodically refreshes workflow status. Meanwhile `worker.main()` polls; `jobs.claim()` atomically leases a job using PostgreSQL locking. A heartbeat extends the lease while work runs.

Durability: The user-facing API does not keep the browser request open for model generation. Job queue, workflow state, event sequence, and graph checkpoint are persisted, so the worker can resume a thread instead of losing the whole workflow on a short-lived request.

3. Worker, graph, routing, and evidence path

Graph stage	Called code / action	State effect and next step
Start → route	<code>WorkflowService.determine_route()</code> → <code>routing.route()</code> , using the ordered policy decision table.	Writes analysis and selected specialist, risk, tool/RAG requirements, review requirement. Unsupported domain completes with a clear abstention.
retrieve	<code>WorkflowService.retrieve_evidence()</code> ; optional selected-observation resolution and ingestion; optional existing analytics query.	Stores retrieval trace/history and tool result D1. No useful evidence and no analytics result ends with a no-evidence response.
generate	<code>WorkflowService.generate_draft()</code> invokes selected specialist(s) in sequence, bounded by the policy iteration limit.	Stores agent output and model-call metadata; specialist must cite available chunks/tool result and may not introduce unsupported numbers.
evaluate	<code>WorkflowService.evaluate_draft()</code> invokes <code>EvaluationService.evaluate()</code> and a separate structured judge call.	Appends a versioned artifact. Failed/weak score requests bounded revision or fails; passing assessment either enters review gate or finalizes if review is not required.
review interrupt	Graph node calls <code>LangGraph interrupt()</code> , with workflow ID, artifact version, and revision.	Execution pauses at the durable checkpoint. Workflow remains <code>AWAITING_HUMAN_REVIEW</code> ; nothing is released as final.

apply review → finalize	Resumed graph verifies the review decision was persisted; approved workflows call finalize().	Only approved current artifact can become COMPLETED. Rejection/revision loops back to generation when allowed.
-------------------------------	---	--

Retrieval and analytics details

- For a selected observation, the service resolves the matching record using the observation intelligence snapshot, then indexes its *original* observation text with provenance such as run, inspection, source file, sheet, and row.
- RetrievalService embeds the query locally and searches one PostgreSQL corpus with vector cosine and PostgreSQL full-text ranking; hybrid mode fuses ranks. A documented keyword fallback is used for embedding/network or vector data failures.
- Retrieval trace records candidates, selected chunk IDs, filters, status, model/version and thresholds. Similarity is a cosine score, not a calibrated probability.
- Questions asking for counts/rates/trends invoke the existing dashboard analytics service; model receives labeled deterministic D1 result, not permission to invent SQL or numerical answers.
- Specialists: EvidenceAgent, InspectionAssessmentAgent, and DataAnalyticsAgent. Combined analytics + observation work is sequential and bounded; no autonomous recursive agent delegation.

4. Generation, evaluation, and human gate

Specialist generation

`backend/agents/specialists.py` loads the generation prompt and calls `StructuredLLM.generate()`. The client restricts generation to a local model name, bounds prompt size, uses a single-call concurrency gate, requests schema-constrained JSON, and validates the resulting Pydantic output. The specialist checks citation IDs and rejects unsupported numerical claims.

Separate judge evaluation

`backend/workflows/evaluation.py` makes a distinct judge-model call using the rubric prompt. Eight dimensions are scored: relevance, groundedness, factual consistency, completeness, domain correctness, safety, evidence coverage, and clarity. The system calculates the mean score and applies configured pass/revision thresholds plus hard minimum groundedness and safety checks. Same-model generator/judge is disclosed because errors may be correlated.

Why the human gate is triggered

- Risk level is HIGH or CRITICAL (mandatory policy floor).
- The user explicitly asks for review; the Evidence panel sets this for selected-observation assessments.
- Retrieval is a WEAK_HIT, or this is already a human-review revision.

On a passing judge result with review required, the artifact is versioned and marked `PENDING`; workflow status becomes `AWAITING_HUMAN_REVIEW`. The graph pauses at its durable interrupt. A draft or passing score alone is not a final response.

Authenticated review and continuation

1. Reviewer obtains pending work from `GET /api/workflows/reviews/pending`, or opens the selected workflow.
2. UI displays the draft, evaluation, citations, retrieved evidence, limitations, routing/risk, and prior review history.
3. Reviewer chooses approve, modify, or reject and provides rationale. API role must be reviewer/admin; the service uses the authenticated principal as authoritative reviewer identity.
4. Request includes the current workflow revision, artifact version, and idempotency key. Server checks role, state, revision, artifact version, and duplicate-key consistency to prevent stale or replayed decisions.
5. Decision and event are saved transactionally; a resume job is queued. Approve/modify marks the artifact approved. Reject can cause a bounded new generation when iterations remain.
6. Worker resumes the LangGraph checkpoint with the persisted review ID. The graph verifies it exists; approval proceeds to finalization. Final response is either the approved artifact or reviewer-modified text.

Modification behavior: Reviewer replacement text gets deterministic checks for unsupported numbers and prohibited conclusions. The current implementation does not call the LLM judge again after a reviewer modification; the authorized reviewer is the approving gate.

Release rule: `finalize()` refuses to publish unless evaluation passed and any required human decision is approved for the current artifact.

5. Debugging: follow one workflow end to end

A. Confirm runtime health (PowerShell)

```
Set-Location "C:\Users\nitis\OneDrive\Desktop\RegInsight-AI"  
docker compose ps  
Invoke-RestMethod http://127.0.0.1:8018/api/health
```

Expect app and worker running, database healthy, and API health to report PostgreSQL plus a loaded dataset. If startup/import is in progress, inspect migration logs; a clean launch still has to load the prepared data.

B. Follow background execution

```
docker compose logs -f worker  
docker compose logs -f app  
docker compose logs --tail=100 migrate  
docker compose top worker
```

Worker logs include structured job failures and `workflow_node` messages with workflow ID, request ID, node/event name, and revision. App security middleware logs HTTP request ID, route, status, user ID, and elapsed time. Do not print `.env` or credentials to logs/chat.

C. Inspect from the application

While a workflow is open, the Evidence panel refreshes every 2.5 seconds. “Open complete operational trace” links to `GET /api/workflows/{id}/trace`, which returns route/risk, retrieval trace/history, D1 tools, agent outputs, artifacts and judge results, human decisions, current status, final response, and ordered events.

Endpoint / observation	Use it to answer
<code>GET /api/workflows/{id}</code>	Current complete workflow state and status.
<code>GET /api/workflows/{id}/trace</code>	Where it routed, what evidence/tools/agents ran, artifacts, review and event sequence.
<code>GET /api/workflows/{id}/retrieval</code>	Retrieved chunks, selected context, hit status and retrieval metadata.
<code>GET /api/workflows/{id}/evaluation</code>	Judge decision, dimensions, issues, rationale, and thresholds for each artifact version.
<code>GET /api/workflows/reviews/pending</code>	Review queue visible to the authenticated user/role.

API access requires a signed-in session. Use the UI or authenticated browser session for these endpoints; do not put access keys in a URL or share them in a report.

AI assessment workflow	Observation classification review
Starts from “AI assessment and workflow review” under a selected Evidence observation. Creates a new workflow and versioned narrative artifact.	Existing “Expert review” form reviews an observation's classification/theme/severity snapshot.
API: /api/workflows. PostgreSQL workflow, graph checkpoint, event, queue, and review decision records.	API: /api/workspace/reviews. Append-only observation decision/revision history.
Authenticated reviewer/admin role is enforced server-side; final narrative remains blocked until approved if gate required.	Reviewer name is self-reported in the current UI; this is not an authenticated electronic signature. Original classification snapshot is preserved.
Approve/modify/reject controls workflow continuation, artifact approval, and final response.	Approve/modify/reject records a label/interpretation decision; it does not approve an AI workflow artifact.

Chatbot boundary: The pre-existing chatbot and investigator are separate application features. The explicit workflow pipeline is not implicitly called for every chat message.

8. Source map: exact call sites

Project file	Entry point / responsibility
frontend/src/reginsight/Evidence.jsx	Selected observation view; embeds workflow panel.
frontend/src/reginsight/WorkflowReview.jsx	Starts request, polls state, renders trace/evaluation, submits review actions.
backend/api/main.py	Application lifespan; registers workflow router; initializes runtime service.
backend/api/security.py; identity.py	Session authentication, request boundaries, reviewer/admin principal.
backend/api/workflows.py	HTTP entry routes: create/read/trace/retrieval/evaluation/pending/review/resume.
backend/workflows/factory.py	Wires policy, local embeddings, retrieval, structured LLM, analytics, repository.
backend/workflows/worker.py; jobs.py	Worker polling, job execution, lease/retry/heartbeat behavior.
backend/workflows/graph.py	LangGraph nodes, conditional transitions, durable human interrupt/resume.
backend/workflows/service.py	Business flow: route, retrieve, generate, evaluate, review, finalize, run/resume.
backend/workflows/routing.py; policy.py	Ordered routing rules, risk/review policy, thresholds and bounded iterations.
backend/rag/service.py; repository.py; chunking.py	Document chunking, local embedding, hybrid retrieval and full evidence trace.

backend/agents/specialists.py; backend/libraries/structured_llm.py	Bounded agent prompts, structured local model calls and citation/numeric checks.
backend/workflows/evaluation.py	Independent judge call, score calculation, hard safety/groundedness gates.
backend/workflows/repository.py; database migrations	Atomic workflow revisions, concurrency checks, event and pending-review persistence.

Quick mental model: React submits; FastAPI authenticates and records; PostgreSQL queues; worker executes; graph checkpoints; routing decides; retrieval/analytics supply evidence; specialists draft; judge evaluates; policy pauses for a human where required; authenticated review persists a decision; worker resumes; finalization enforces the approval gate.

This guide documents current source behavior, not regulatory certification. Scores and local model outputs are decision support only. Always inspect source evidence and have a qualified human reviewer make regulatory decisions.